

Lecture 18:

Deep Learning

Convolutional Neural Networks

ML/AI for Engineering Applications
ENV 6932

Sara Behdad
Fall 2025

Outline

- **Recall:** Activation Functions, Loss Functions, Gradient Descent, Back Propagation, Bias-Variance in NNs
- Convolutional Neural Networks

Types of Neural Networks

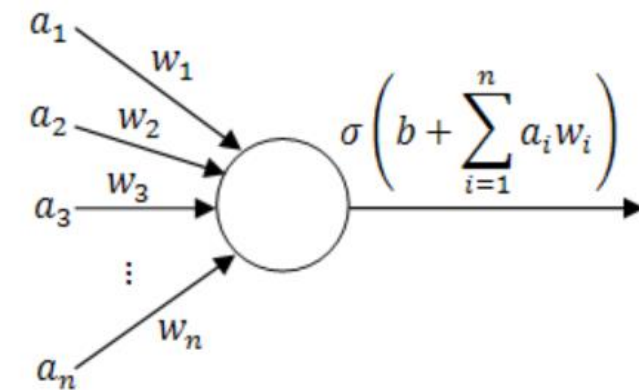
- **1. Feedforward Neural Network**

A network where connections between the units do *not* form a cycle, the info moves forward, from the input nodes to the output nodes.

1.1. Single-layer Perceptron

1.2. Multi-layer perceptron (MLP)

1.3. Convolutional Neural Network (CNN)



Simple Perceptron

Types of Neural Networks

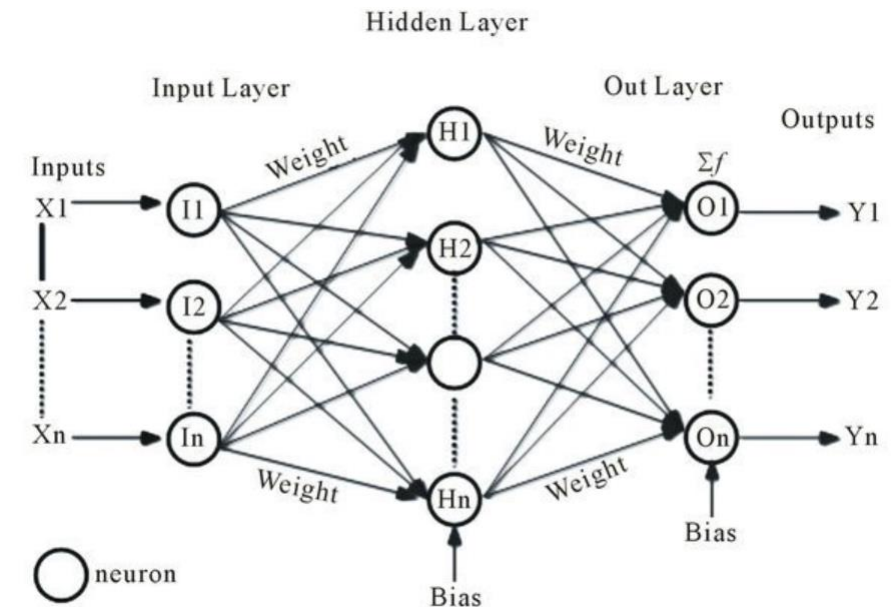
- **1. Feedforward Neural Network**

A network where connections between the units do *not* form a cycle, the info moves forward, from the input nodes to the output nodes.

1.1. Single-layer Perceptron

1.2. Multi-layer perceptron (MLP)

1.3. Convolutional Neural Network (CNN)



Types of Neural Networks

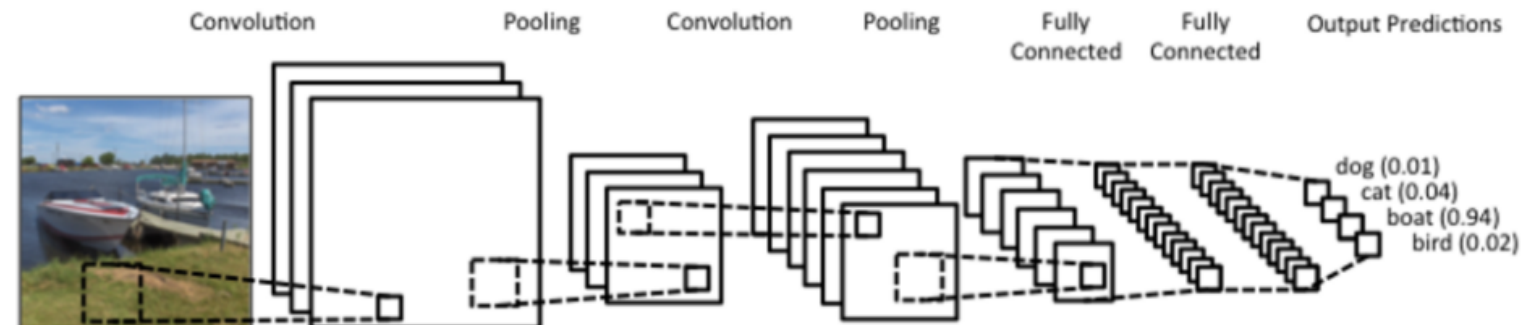
- **1. Feedforward Neural Network**

A network where connections between the units do *not* form a cycle, the info moves forward, from the input nodes to the output nodes.

1.1. Single-layer Perceptron

1.2. Multi-layer perceptron (MLP)

1.3. Convolutional Neural Network (CNN) [CNN are neural networks specifically for images]

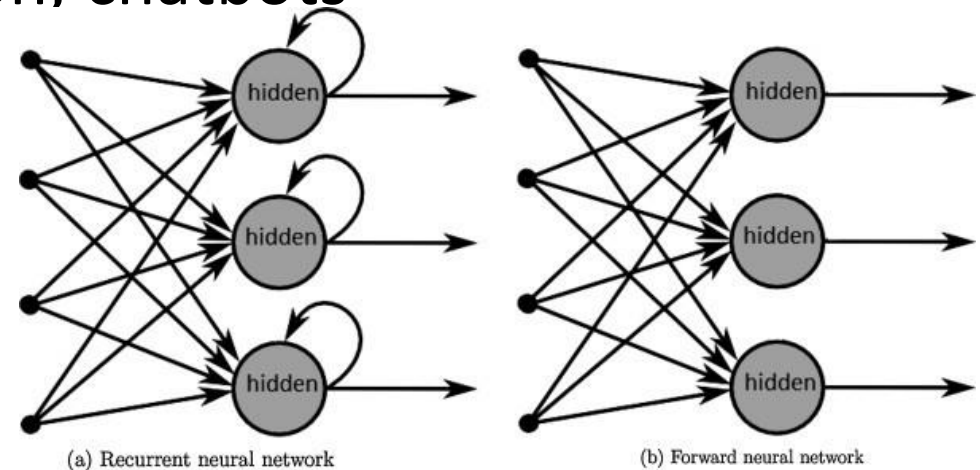


Types of Neural Networks

- **2. Recurrent neural networks**

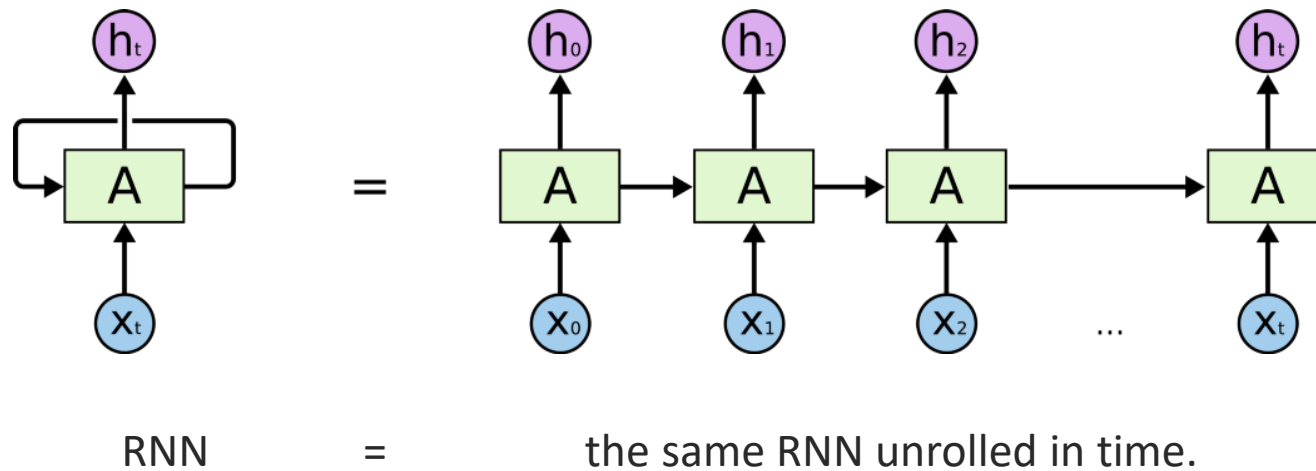
In RNN, connections form a directed cycle (data are propagated forward, and also backwards). RNNs can use their memory to process arbitrary sequences of inputs.

- **Applications:** text generation, machine translation, speech recognition, generating image description, chatbots



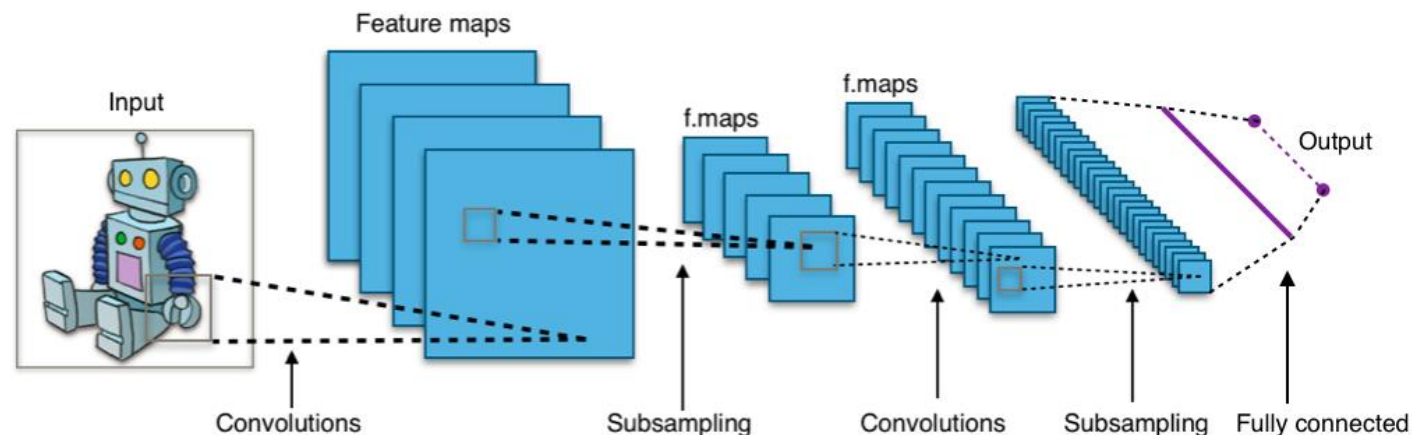
Types of Neural Networks

- 2. Recurrent neural networks



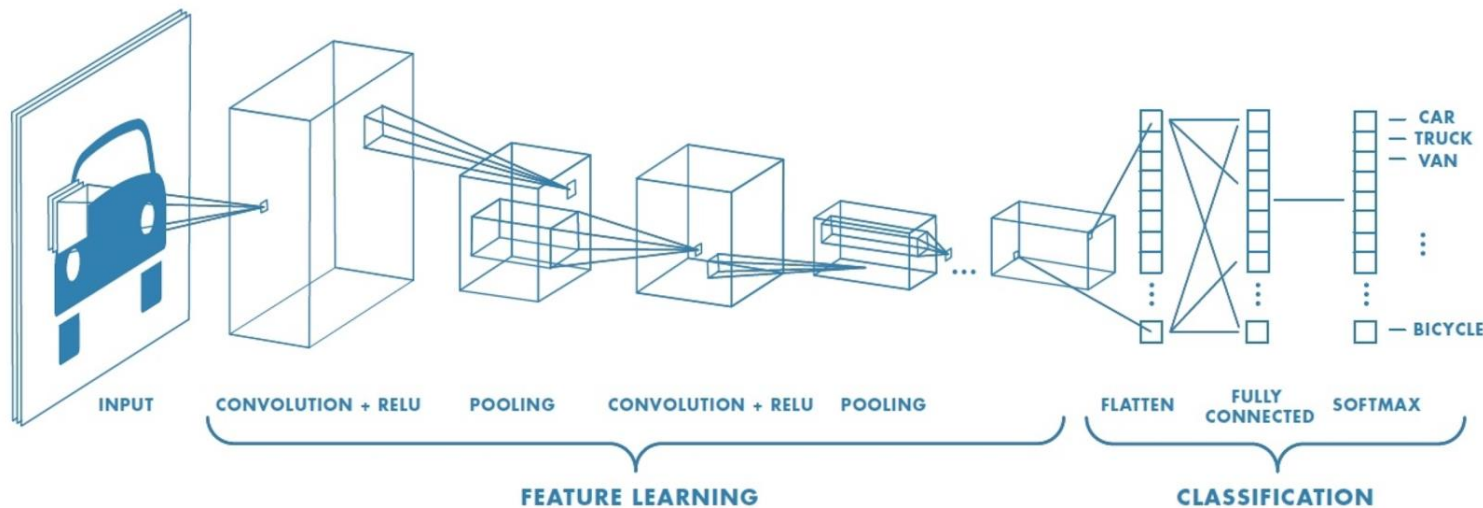
CNNs: Convolutional Neural Networks

- A special type of neural network that has a known, grid-like topology.
- **Examples:** (1) time-series data (can be represented as a 1D grid), (2) image data (2D grid of pixels).
- CNN employs a mathematical operation called **convolution** (a specialized kind of linear operation).



CNNs: Convolutional Neural Networks

- CNN consists of two basic building blocks:
- **The Convolution Block:** Contains the Convolution Layer and the Pooling Layer. It performs *Feature-Extraction*
- **The Fully Connected Block:** Contains a fully connected simple neural network architecture. It performs the *Classification* task.



Defining the CNN layers:

(1) CONVOLUTIONAL LAYER:

- The Convolutional Layer is related to **feature extraction**.
- First let's define the idea of '*filters*'.
- **Filters:** Filters or 'kernels' are also an image that depict a particular feature.
- **Example of filter:** determine whether the curve presents in an image.

0	0	0	0	0	30	0
0	0	0	0	30	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	0	0	0	0

Pixel representation of filter



Visualization of a curve detector filter

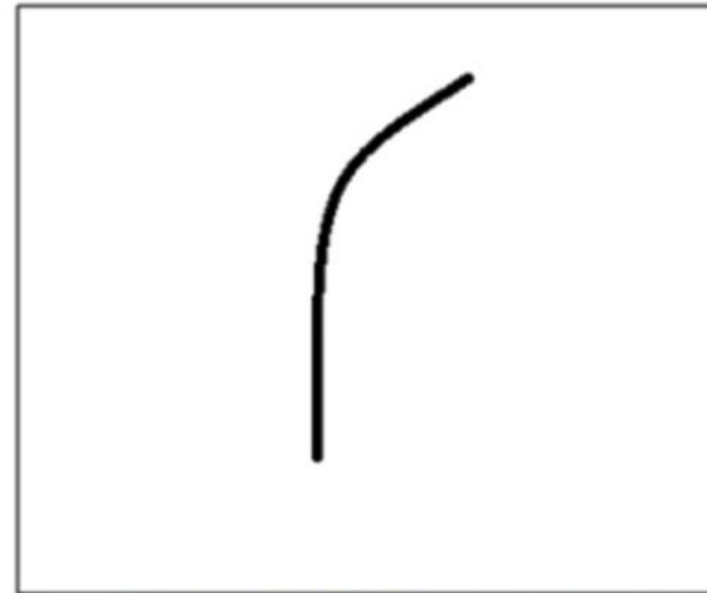
Defining the CNN layers:

(1) CONVOLUTIONAL LAYER:

- **Example of filter:** determine whether the curve presents in an image.

0	0	0	0	0	30	0
0	0	0	0	30	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	0	0	0	0

Pixel representation of filter



Visualization of a curve detector filter

Defining the CNN layers:

(1) CONVOLUTIONAL LAYER:

- **Convolution:** It is a special operation applied on a particular matrix (the input image matrix) using another matrix (the filter-matrix).
- The operation includes **multiplying the image matrix, with the filter matrix.**
- **This is done for the values of all the cells within the span of the filter matrix and are added together to generate an output.**

Defining the CNN layers:

(1) CONVOLUTIONAL LAYER:

- **Example:**
- Image matrix, $A = [2 \ 5 \ 17]$
- Filter matrix, $B = [1 \ 0 \ 1]$.
- Then $A \text{ *(convolve) } B = [2*1 + 5*0 + 17*1] = [2 + 17] = [19]$.
- In practice, both of the matrix are 2-D.

Defining the CNN layers:

(1) CONVOLUTIONAL LAYER:

- **Example** of how convolution works

1 _{x1}	1 _{x0}	1 _{x1}	0	0
0 _{x0}	1 _{x1}	1 _{x0}	1	0
0 _{x1}	0 _{x0}	1 _{x1}	1	1
0	0	1	1	0
0	1	1	0	0

Image

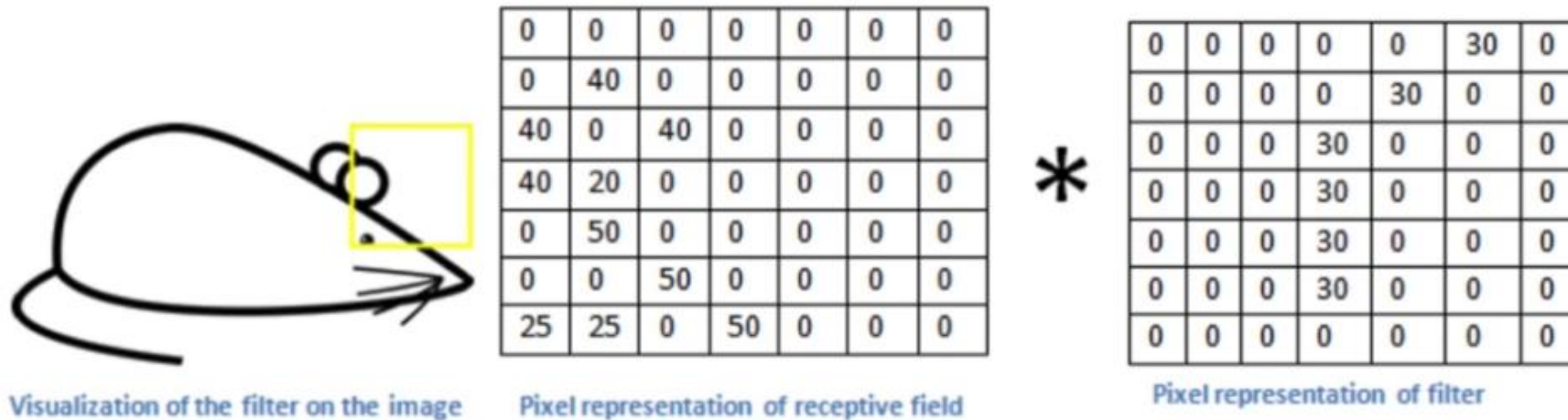
4		

Convolved
Feature

Defining the CNN layers:

(1) CONVOLUTIONAL LAYER:

- now, how do we find a particular feature?
- Sample #1:

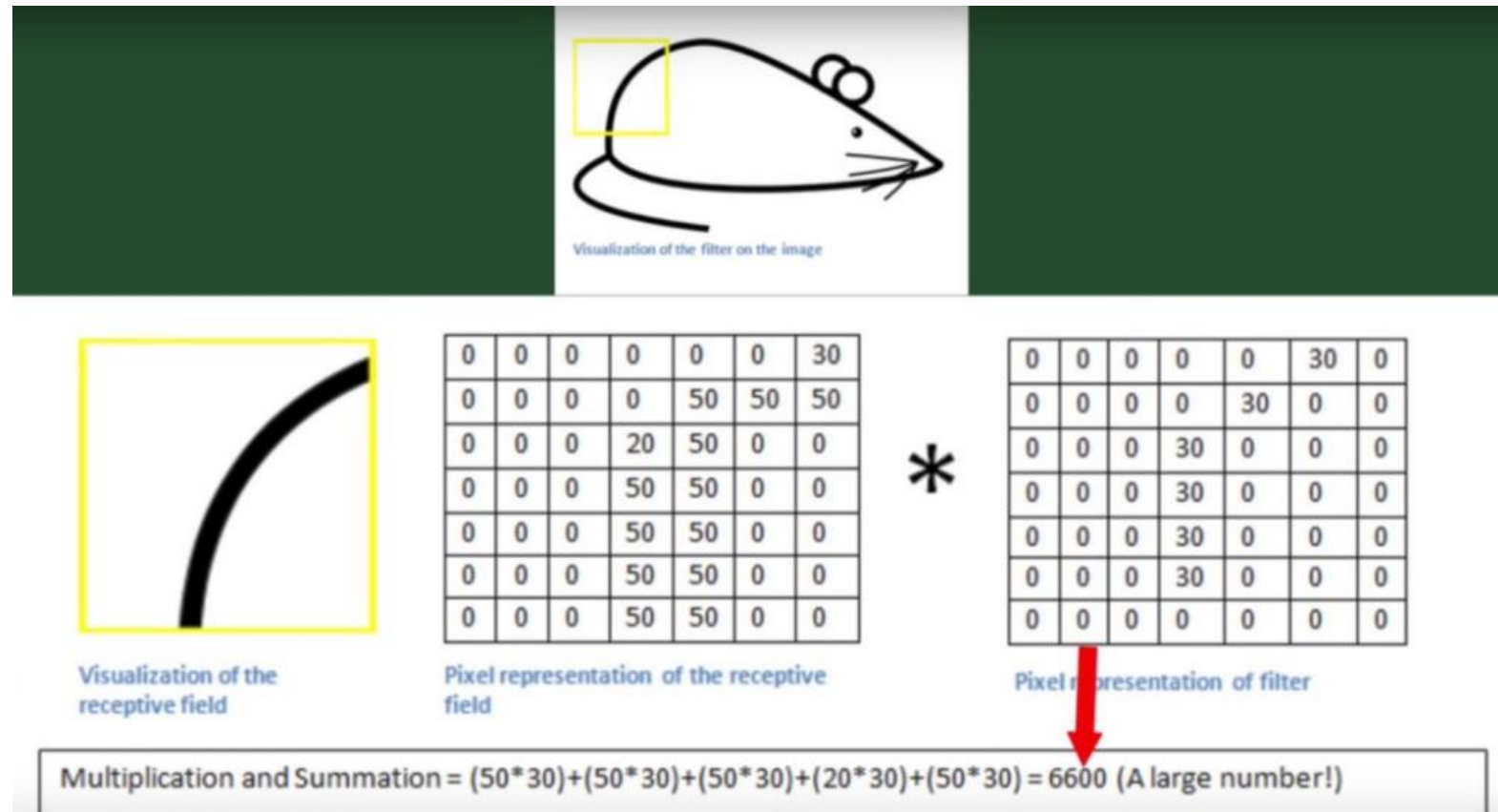


Multiplication and Summation = 0

Defining the CNN layers:

(1) CONVOLUTIONAL LAYER:

- Sample #2:



Defining the CNN layers:

(1) CONVOLUTIONAL LAYER:

- **Intuition:** if the feature exists in a particular part of the image, it returns a very large value on convolution, otherwise, return a small value. (e.g. 0 vs 6600)

Defining the CNN layers:

(1) CONVOLUTIONAL LAYER:

- **Padding**
- The no of times, the cells lying within the matrix is considered for the operation is more than the no. of times, the values of cells in the corners or at the borders, are considered.
- **Recall**

1 _{x1}	1 _{x0}	1 _{x1}	0	0
0 _{x0}	1 _{x1}	1 _{x0}	1	0
0 _{x1}	0 _{x0}	1 _{x1}	1	1
0	0	1	1	0
0	1	1	0	0

Image

4		

Convolved
Feature

Defining the CNN layers:

(1) CONVOLUTIONAL LAYER:

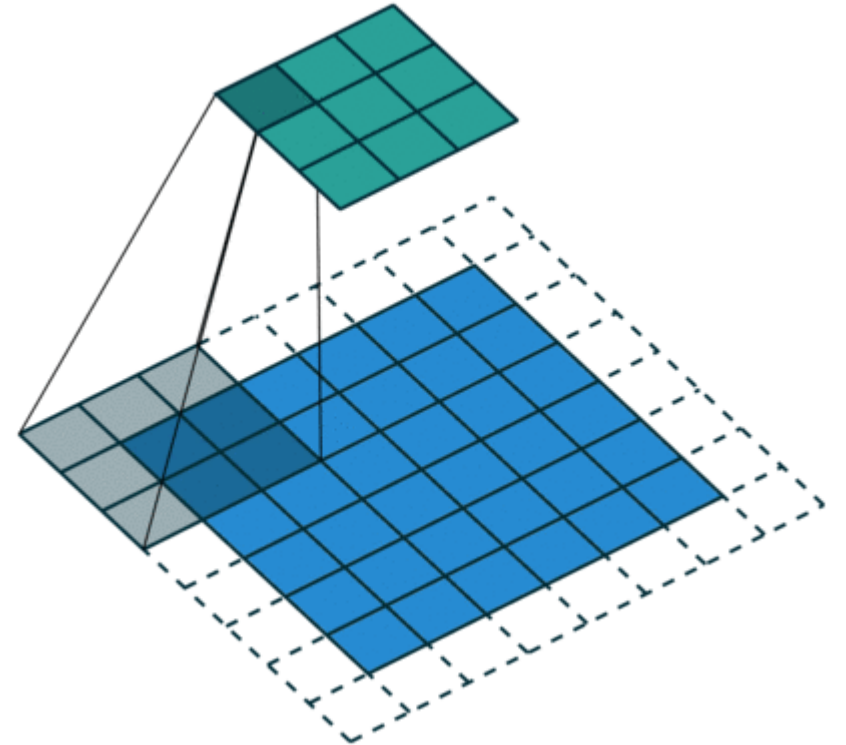
- **Padding**
- **Solution:** A padding of 0's around the actual matrix.
- Adding '0' wouldn't supply any extra info, but will help considering the previously less-accounted cells.

0	0	0	0	0	0
0	35	19	25	6	0
0	13	22	16	53	0
0	4	3	7	10	0
0	9	8	1	3	0
0	0	0	0	0	0

Defining the CNN layers:

(1) CONVOLUTIONAL LAYER:

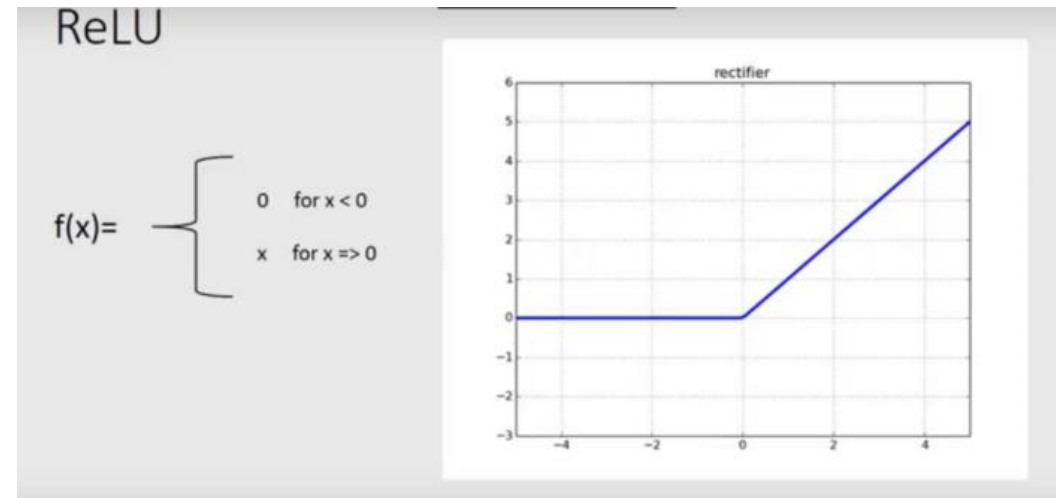
- **Striding**
- In 'strided' convolution: instead of shifting the filter one-row or one-column at a time, shift it, maybe, 2 or 3 rows or columns, each time.
- It reduces the computation and the size of the output matrix.
- In large images, this doesn't lose data, but reduces computation.



Defining the CNN layers:

(1) CONVOLUTIONAL LAYER:

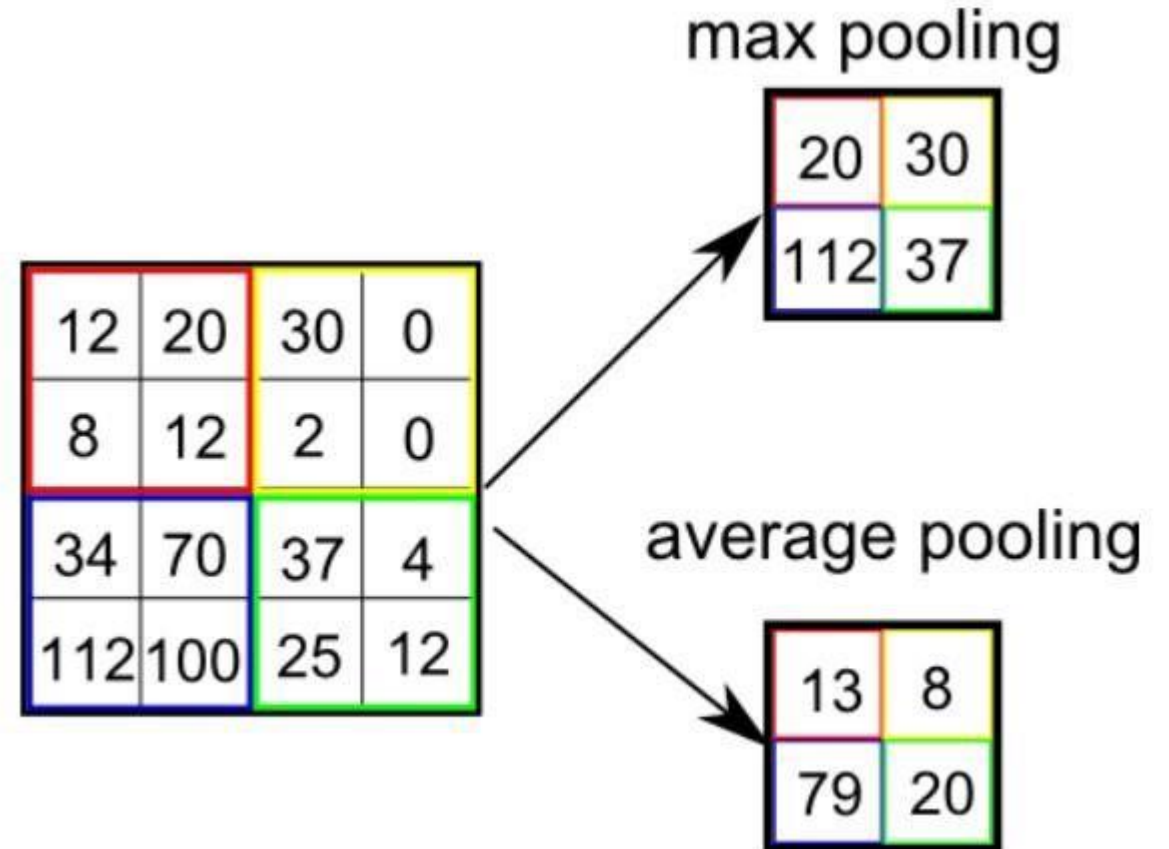
- **RELU Activation:** *RELU* is applied on all the cells of all the output-matrix.
- **The intuition:** after convolution, if a particular function outcomes '0' or a negative value, it implies that the feature does not exist there and we denote it by '0'.



Defining the CNN layers:

(2) POOLING LAYER:

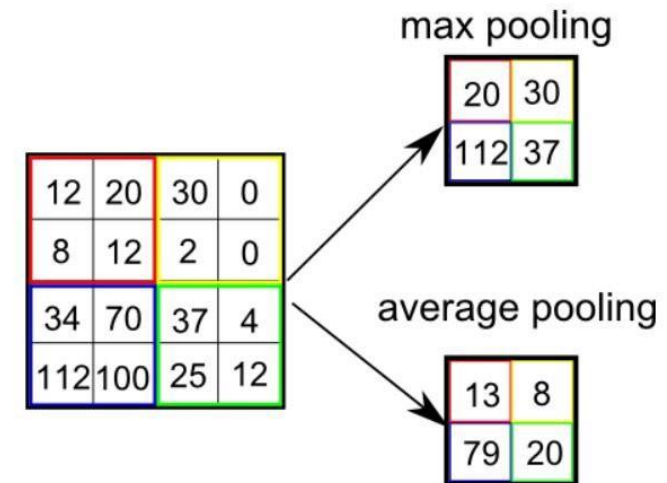
- This layer consists of extracting a particular value from a set of values (usually the max or the average value) and reduces the size of the output matrix.
- For example, in MAX-POOLING, the **max value among all the values of say a 2 X 2 part of the matrix is selected.**



Defining the CNN layers:

(2) POOLING LAYER:

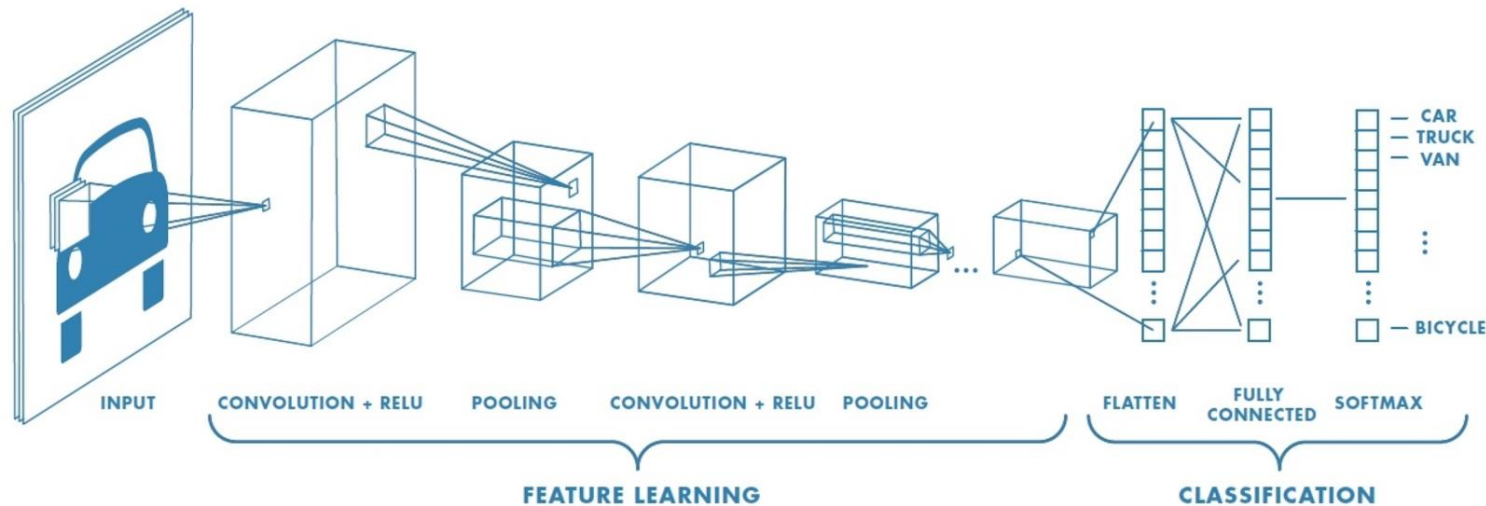
- Thus, we are actually taking in the values denoting the presence of a feature in that section of the image and getting rid of unwanted information.
- It is common to periodically insert a Pooling layer in-between successive convolutional blocks to reduce the number of parameters and computation in the network.



Defining the CNN layers:

CONVOLUTIONAL AND POOLING LAYER:

- The **CONVOLUTIONAL** and the **POOLING LAYER together** form the **CONVOLUTIONAL BLOCK** of the CNN architecture.
- A simple CNN often consists of a minimum of three of these Convolutional Block, that performs feature extraction at various levels.

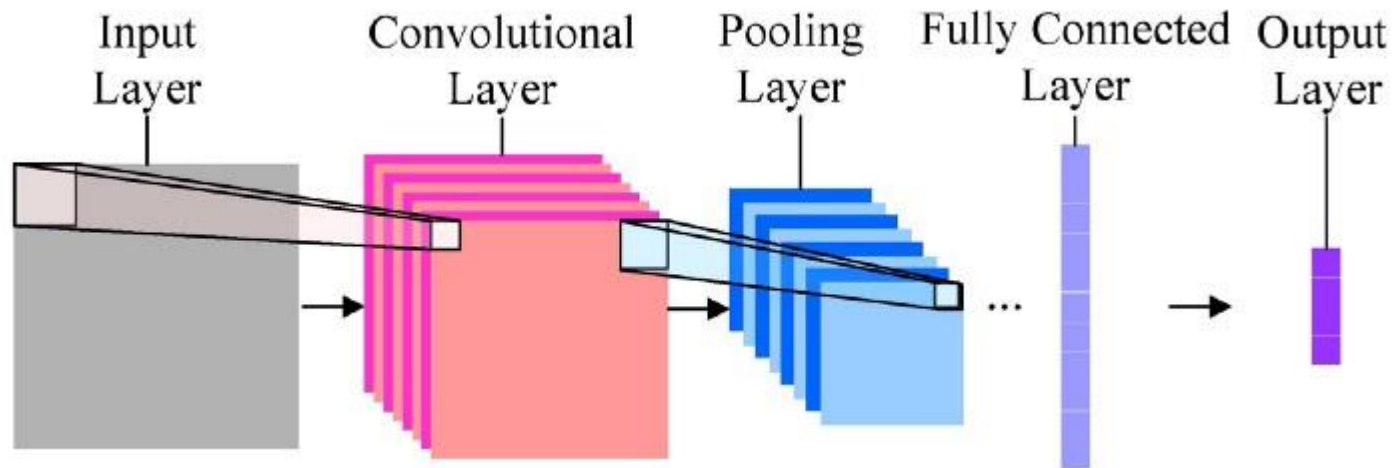


Defining the CNN layers:

- **(3) FULLY CONNECTED LAYER:**
- This layer forms the last block of the CNN, related to *classification*.
- It is essentially a Fully connected Simple Neural Network, consisting of two or three hidden layers and an output layer generally implemented using 'Softmax Regression'.
- It performs classification among a large no of categories.

Bringing Everything Together

- **(3) FULLY CONNECTED LAYER:**
- This layer forms the last block of the CNN, related to *classification*.
- It is essentially a Fully connected Simple Neural Network, consisting of two or three hidden layers and an output layer generally implemented using 'Softmax Regression'.
- It performs classification among a large no of categories.



Softmax Regression

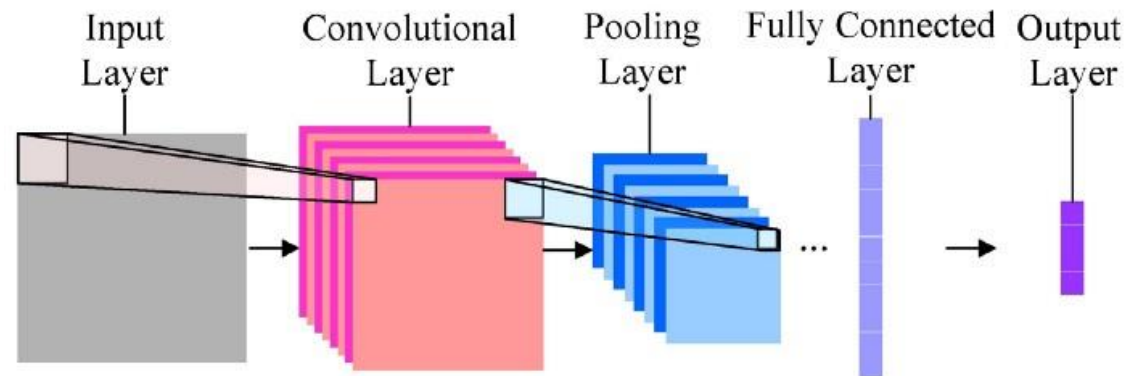
- Softmax (or multinomial logistic regression) is a generalization of logistic regression to handle multiple classes.
- In logistic regression the labels are binary: $y(i) \in \{0, 1\}$.
- Softmax regression allows to handle $y(i) \in \{1, \dots, K\}$, K classes.

$$h_{\theta}(x) = \frac{1}{1 + \exp(-\theta^{\top} x)},$$

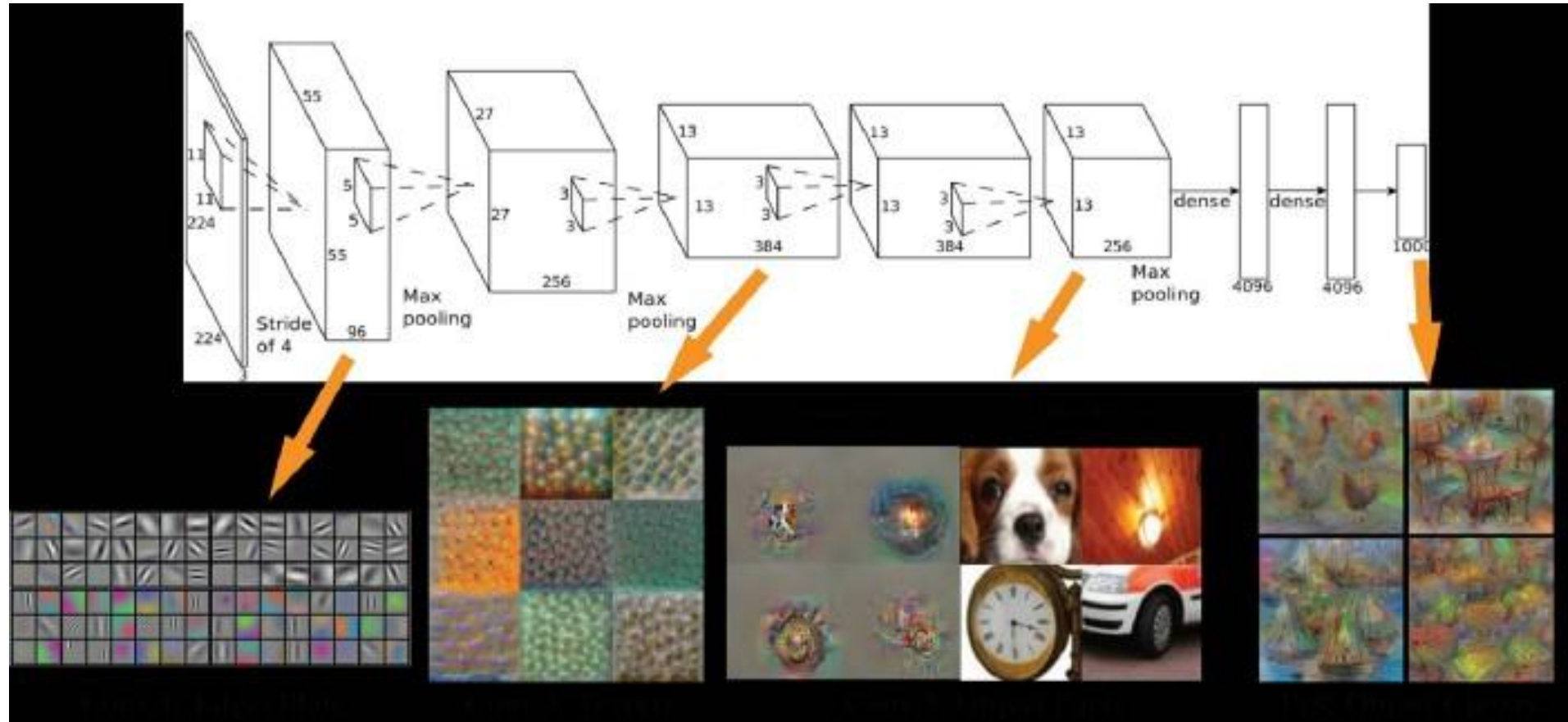
$$J(\theta) = - \left[\sum_{i=1}^m y^{(i)} \log h_{\theta}(x^{(i)}) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)})) \right]$$

A General Question

- How to define the matrix for each filter?
- The CNN is trained over a larger no of images during the training phase and for each time, the error generated is fed back to adjust the values of the matrices in each layer, similar to training of a Simple Neural Network.
- It is through **BACKPROPAGATION**.



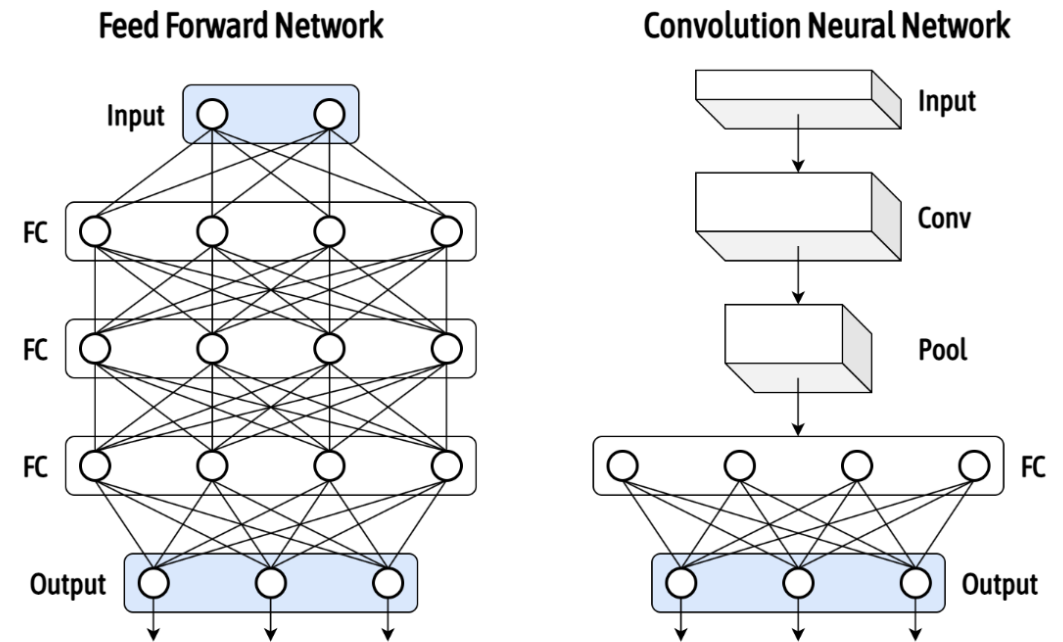
Visualization of what is happening



The visualization obtained at each layers in all the levels.

Why CNNs?

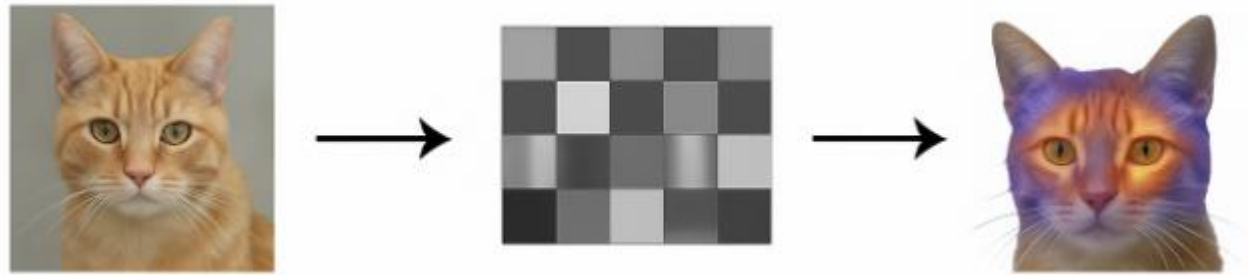
- Traditional fully connected networks treat all input pixels equally, therefore, they lose spatial information.
- Images have local patterns (edges, corners, textures).
- CNNs exploit local connectivity and parameter sharing.
- CNNs are translation-invariant and biologically inspired (visual cortex).



<https://wikidocs.net/165403>

Intuition Behind CNNs

- CNNs “scan” the image with small filters.
- Each filter detects one kind of feature (e.g., vertical edge).
- Later layers combine features ---- complex patterns ---- objects.
- Hierarchical representation:
 - Layer 1 detects: edges
 - Layer 2 : textures
 - Layer 3 : object parts



CNNs learn **hierarchical representations**:

Edges → Textures → Object Parts → Object Categories

Hierarchical Feature Learning in CNNs

Layer 1: Low-Level Features (Edges and Colors)

- Filters detect **simple visual patterns**:
 - Edges (horizontal, vertical, diagonal)
 - Corners, color gradients, brightness contrast
- Focus on **pixel-level details**, acting as early 'edge detectors.'

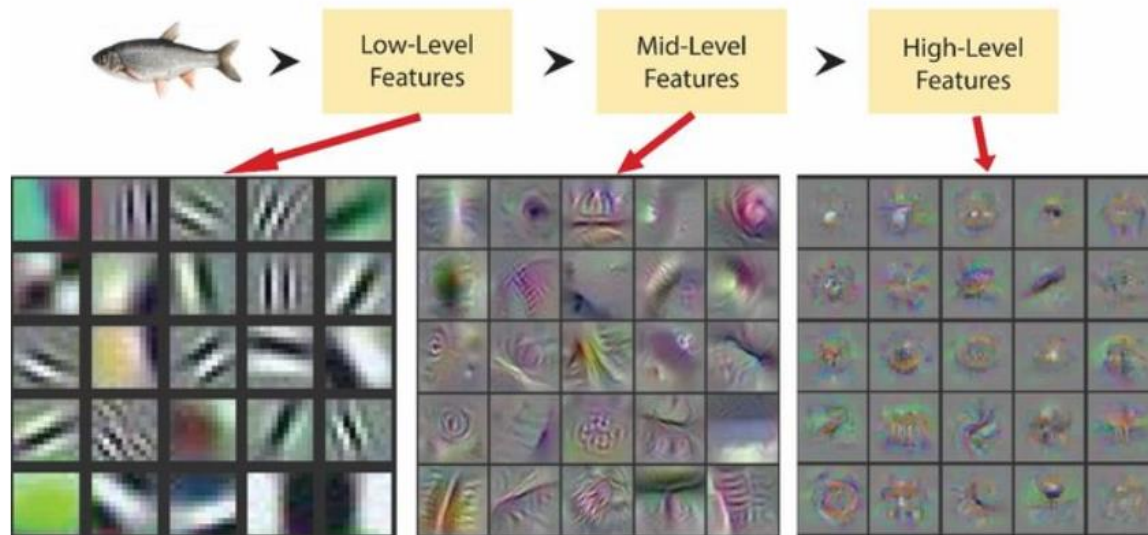
Layer 2: Mid-Level Features (Textures and Motifs)

- Combines simple edges into **patterns and textures**:
 - Stripes, dots, curves, fur, wheels
- Represents **groups of edges** forming localized structures.
- Begins capturing **shapes and orientations**.

Hierarchical Feature Learning in CNNs

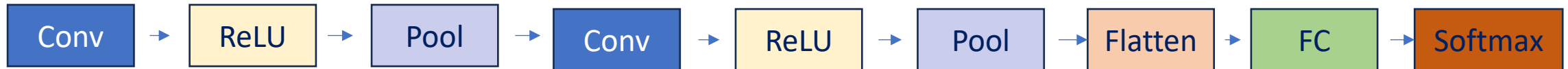
Layer 3 and Deeper: High-Level Features (Object Parts and Whole Objects)

- Combines textures into **semantic parts**:
 - Eyes, noses, leaves, car wheels
- Later layers respond to **entire objects or classes**.
- CNN transitions from *what the image looks like to what it is*.



CNN Architecture Overview

- CNNs consist of two parts:
 - **Feature Extraction:** Convolution + Pooling Layers
 - **Classification:** Fully Connected Layers
- Typical structure:

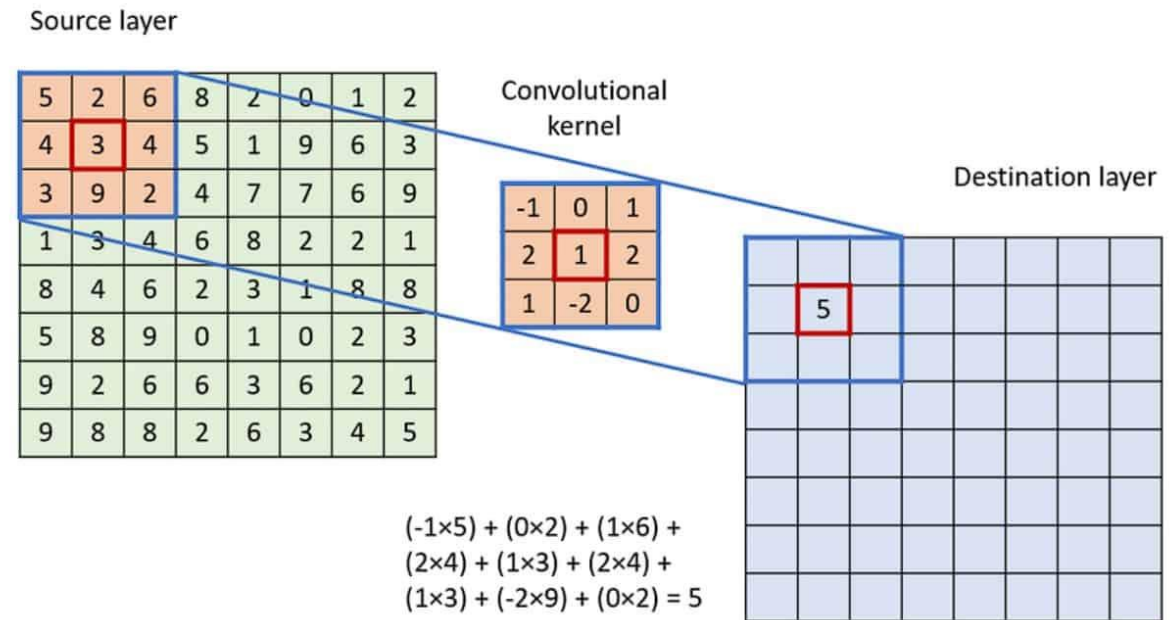


- Each layer reduces spatial size and increases depth.

1. Convolution Operation

- Convolution = weighted sum of pixel neighborhoods.
- Each filter/kernel slides over image and computes feature map.
- Example:

$$S(i, j) = \sum_m \sum_n X(i + m, j + n) K(m, n)$$



Stride, Padding, and Dilation in CNNs

Concept	Description	Effect on Output
Stride (S)	How far the filter moves at each step (row/column).	Larger stride, smaller output.
Padding (P)	Zeros added around image borders before convolution.	Keeps output size same or increases it
Dilation (D)	Adds gaps between filter elements.	Expands the receptive field, captures wider context without increasing parameters.

Formula for Output Size (Size of each feature map)

Where:

- **W** = input size (width or height)
- **K** = kernel (filter) size
- **P** = padding
- **S** = stride

$$\text{Output Size} = \frac{(W - K + 2P)}{S} + 1$$

Formula for Output Size

Given Parameters:

- Input size $W = 7$
- Kernel size $K = 3$
- Padding $P = 1$
- Stride $S = 1$

$$\text{Output Size} = \frac{(W - K + 2P)}{S} + 1$$

$$\text{Output} = \frac{(7 - 3 + 2 \times 1)}{1} + 1 = 7$$

Result: Same size (7×7) because padding compensates for kernel size.

Understanding Main CNN Parameters

Parameter	Symbol	Meaning / Function	Effect on Feature Map
Input Size	(W)	Spatial dimension of the input image or feature map (e.g., 32×32).	Larger input, therefore larger output.
Kernel Size (Filter Size)	(K)	Size of the sliding filter (e.g., 3×3, 5×5). Determines how much local area is seen at once.	Larger kernel, therefore smaller output; captures broader content.
Padding	(P)	Number of zeros added around the image borders. Preserves edge information.	More padding, therefore larger (or same-sized) output.
Stride	(S)	Step size for sliding the filter within the input.	Larger stride, so smaller output.
Dilation	(D)	Adds spacing between kernel elements.	Expands receptive field without increasing kernel size.

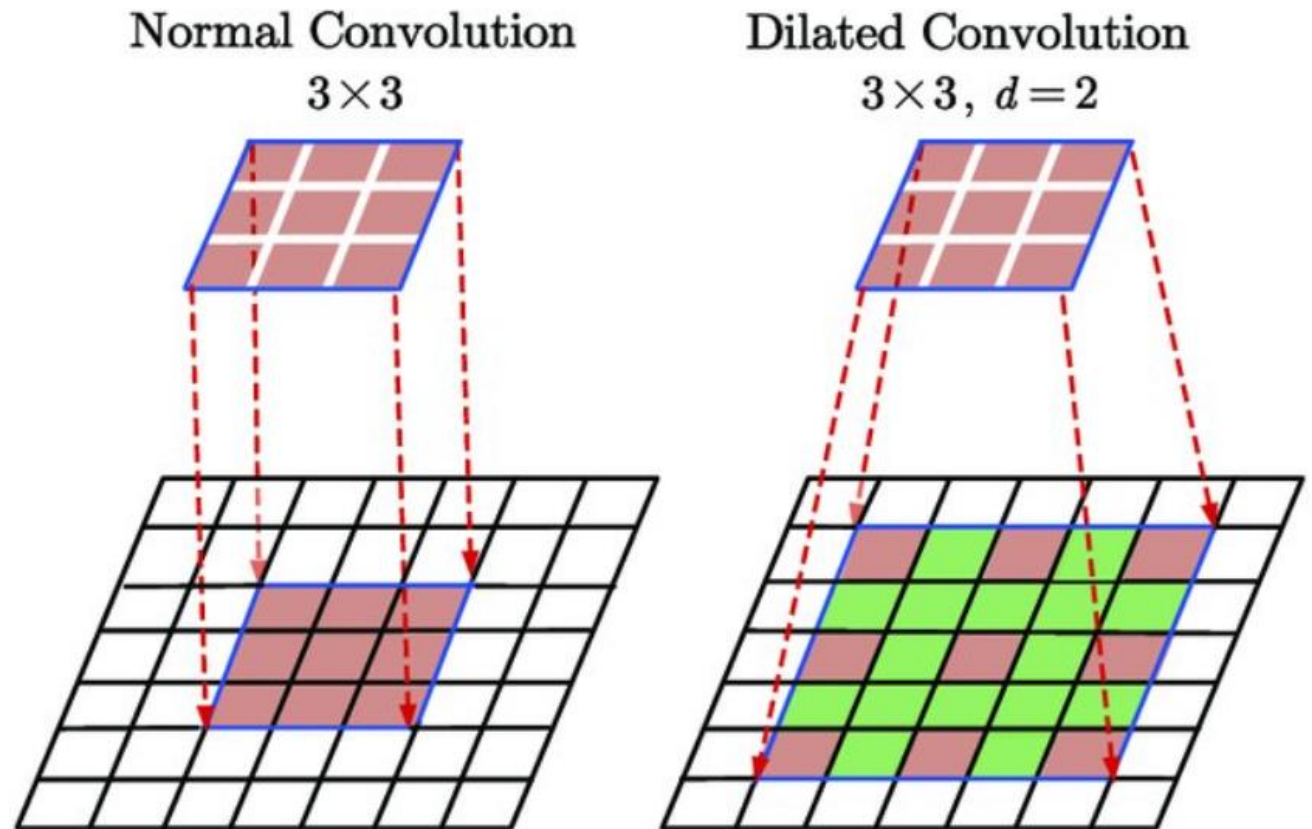
Example, Continued

- Modify the Parameters:

Case	(P)	(S)	Output Calculation	Result
A. "Same" Padding	1	1	$(7-3+2\times 1)/1 + 1 = \mathbf{7}$	Output same size
B. No Padding	0	1	$(7-3+0)/1 + 1 = \mathbf{5}$	Output smaller
C. Stride 2	1	2	$(7-3+2)/2 + 1 = \mathbf{4}$	Downsampled output

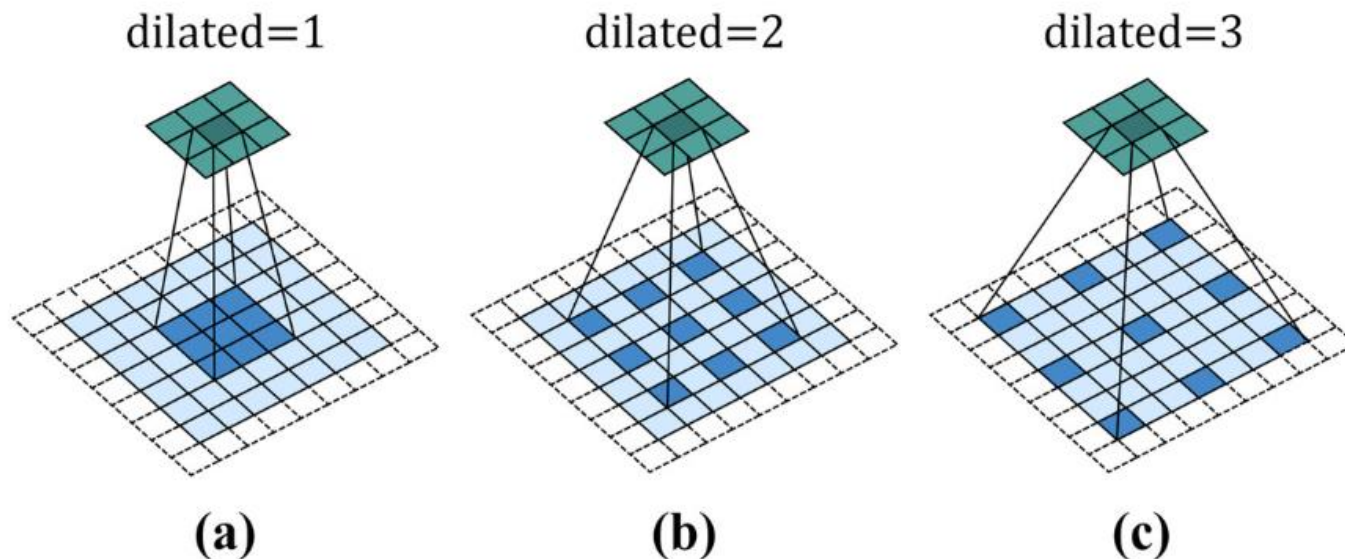
What is Dilation?

- Dilation (D) controls the spacing between kernel elements in a convolution filter.
- A dilated convolution “stretches” the kernel. It skips over input pixels with gaps.
- This increases the receptive field (area of input each neuron sees)



How Dilation Works

- In a standard convolution (dilation = 1):
kernel covers *contiguous* pixels.
- In a dilated convolution (dilation = 2):
kernel elements are spaced by **1 zero** (gap of 1 pixel).
- In general, spacing between kernel cells = **dilation – 1**



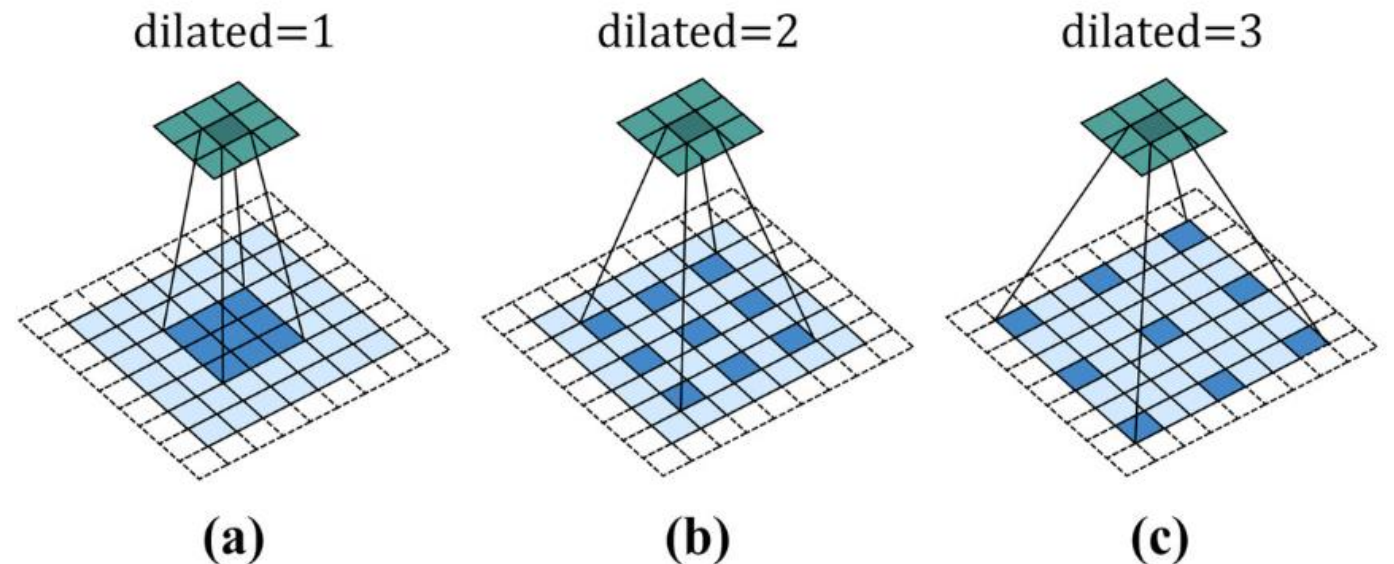
How Dilation Works

Example:

- A 3×3 kernel with **D = 1** covers 3×3 area.
- With **D = 2** covers a 5×5 effective area.
- With **D = 3** covers a 7×7 effective area.

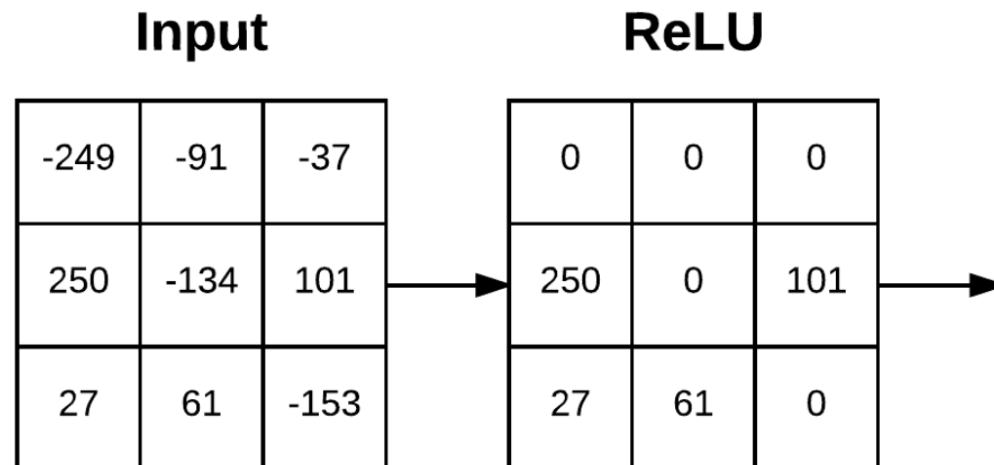
Kernel Size Formula:

$$K_{eff} = K + (K - 1)(D - 1)$$



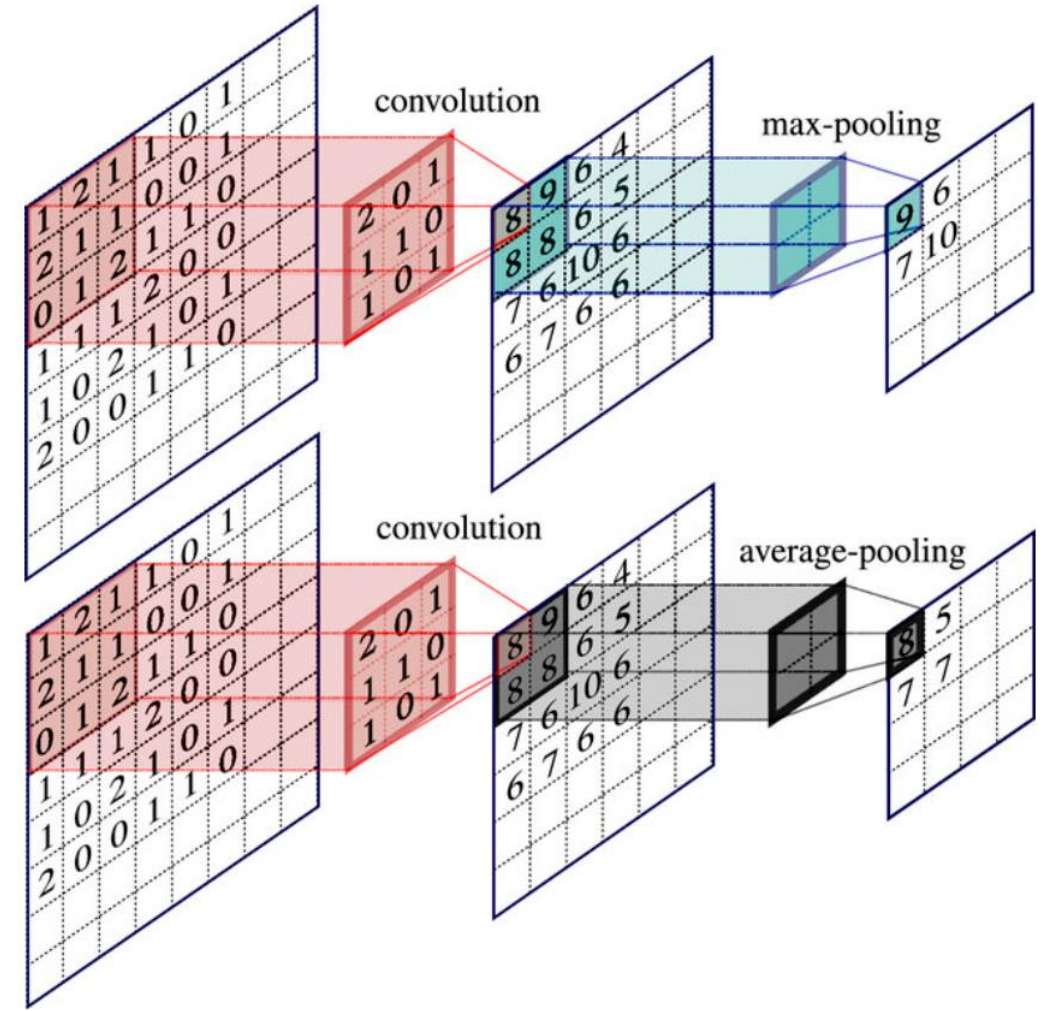
2. ReLU Activation

- After convolution, apply ReLU (Rectified Linear Unit):
 $f(x) = \max(0, x)$
- Removes negative values, therefore, sparsity and faster training.
- Keeps positive signals only (nonlinear transformation).



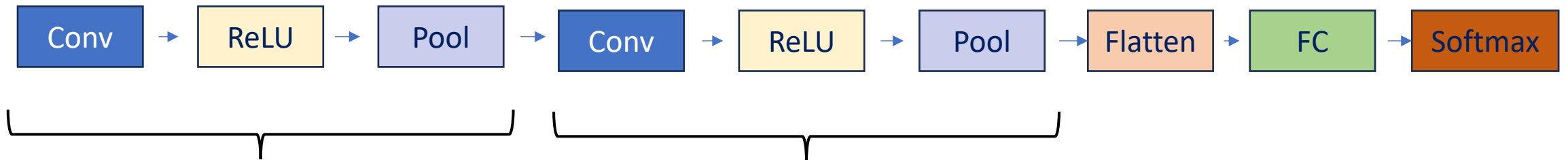
3. Pooling Layer

- Reduces spatial size (downsampling).
- Keeps dominant features and makes model invariant to small shifts.
- Common types:
 - Max Pooling (most common)
 - Average Pooling
- Example: 2×2 Max Pool halves image size.



Convolutional Block

- One Conv Block = Convolution + ReLU + Pooling
- Extracts progressively complex features.
- CNNs often have 2-5 convolutional blocks stacked.



Training a CNN

1. Forward Pass:

- Compute convolutions, activations, pooling, FC outputs.

2. Loss Computation:

- Compare prediction vs. label (e.g., cross-entropy loss).

3. Backward Pass (Backpropagation):

- Compute gradients w.r.t weights via chain rule.

4. Parameter Update:

- Use optimizer (SGD, Adam) to adjust filters.

Understanding Parameters in Neural Networks

- Parameters are **learnable values**: *weights and biases* which are updated during training.
- In a CNN, parameters come from filters (kernels) applied to local image regions.
- Each filter learns to detect a specific feature (edge, texture, etc.).
- The same filter is reused through the image: **weight sharing**.

Parameters in Convolutional Layer

- **Equation:**

$$\text{Parameters per Conv layer} = (K_H \times K_W \times C_{in}) \times C_{out} + C_{out}$$

- **Where:**

- K_H, K_W : kernel height and width
- C_{in} : input channels
- C_{out} : number of filters (output channels)

Parameters in Fully Connected (FC) Layers

- Each neuron connects to **every input value**: no weight sharing.
- Parameters = all input connections + 1 bias per neuron.

- **Equation:**

$$\text{Parameters per FC layer} = (N_{in} \times N_{out}) + N_{out}$$

- **Where:**

- N_{in} : number of inputs
 - N_{out} : number of neurons (outputs)
- FC layers ignore spatial structure: treat all pixels as independent inputs.

Example: Conv vs. FC Parameters

- Input:
 - Image = 32×32×3 or 3,072 input values
 - CNN layer = 16 filters, each 4×4×3
 - FC layer = 16 output neurons

- **Convolutional Layer**

$$(K_H \times K_W \times C_{in}) \times C_{out} + C_{out}$$

$$(4 \times 4 \times 3) \times 16 + 16 = 768 + 16 = \mathbf{784 \text{ parameters}}$$

- **Fully Connected Layer**

$$(N_{in} \times N_{out}) + N_{out}$$

$$(3,072 \times 16) + 16 = \mathbf{49,168 \text{ parameters}}$$

Example: Conv vs. FC Parameters

Property	Convolutional (4×4×3)	Fully Connected (Dense)
Input size	32×32×3	3,072 values
Output	32×32×16** feature maps	1×16* vector
Parameters	784	49,168
Connectivity	Local (4×4 region)	Global (all pixels)

****The user (designer) defines the number of feature maps!**

It's a **hyperparameter**, not learned from data.

It is chosen based on:

- **Model complexity desired**
- **Size of dataset**
- **Hardware limits (GPU memory)**
- **Depth of the network**

*If you are classifying **16 categories**, one neuron per class.

Recall: the size of each map is calculated using formula.

Typical CNN design pattern

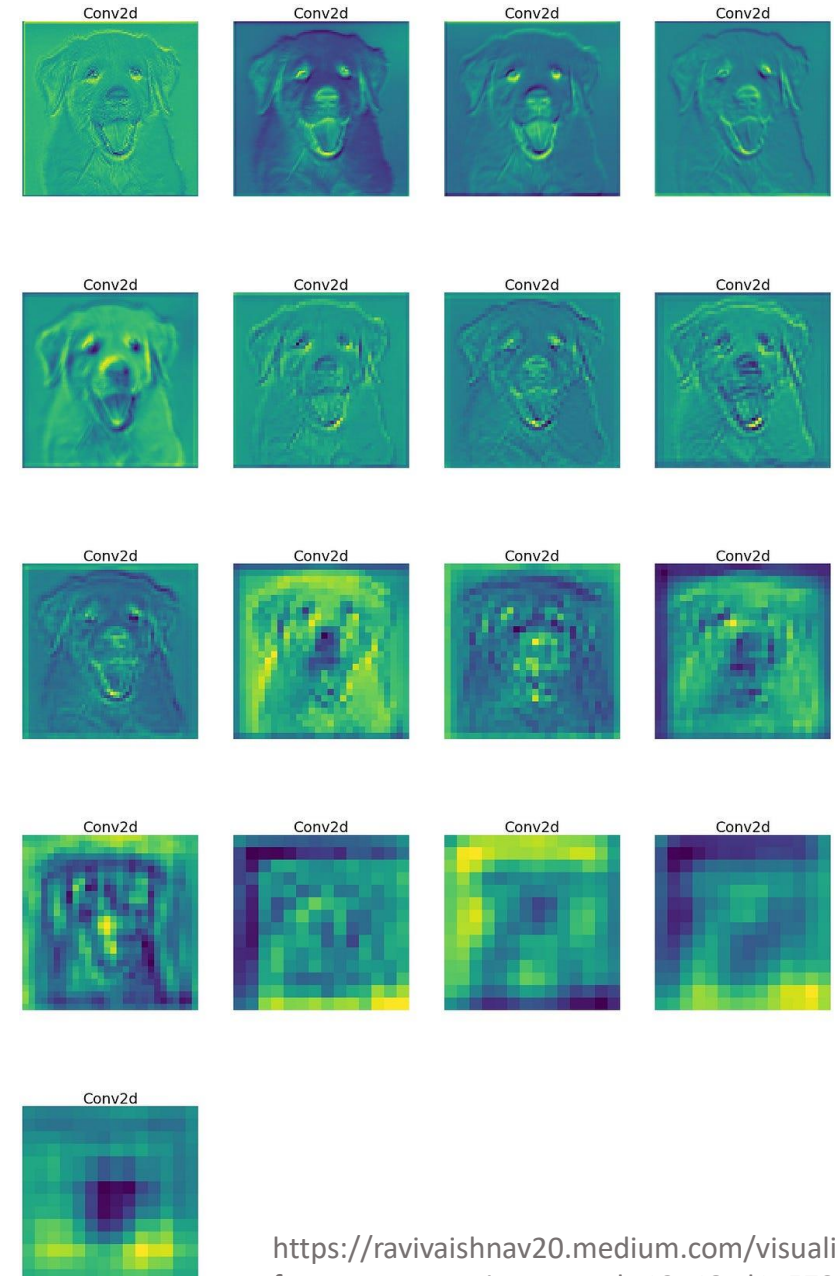
Layer	# of Filters	Reason
Conv1	16 or 32	Small, therefore capture basic edges/textures
Conv2	32 or 64	More, therefore detect combinations of simple features
Conv3	64-128	Even more, therefore detect complex shapes/parts
FC	128-1024	Combine all features before output layer
Output Layer	# of classes (e.g., 10 for CIFAR-10)	Classification scores

Relationship Between Filters and Feature Maps

- A filter and a feature map are not the same object, but they are one-to-one related in a convolutional layer.
- **Filters (a.k.a. Kernels)**
 - Are **learnable weight tensors** (e.g., $4 \times 4 \times 3$).
 - They are part of the model parameters: what the CNN *learns during training*.
 - Each filter detects a specific type of feature (edge, texture, etc.).
- **Feature Maps (a.k.a. Output Maps)**
 - Are the outputs produced when a filter is applied to the input.
 - They depend on both the filter weights and the input image values.
 - These are *not parameters*: they are *computed activations*.

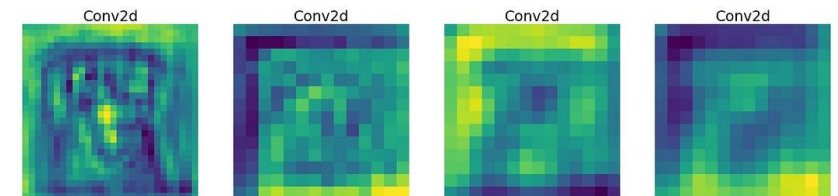
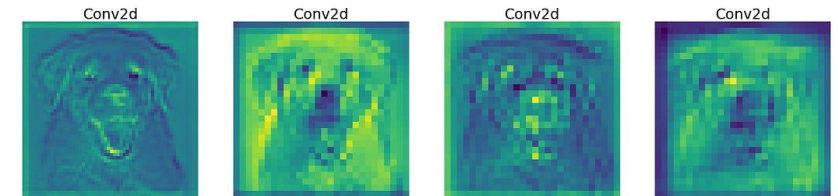
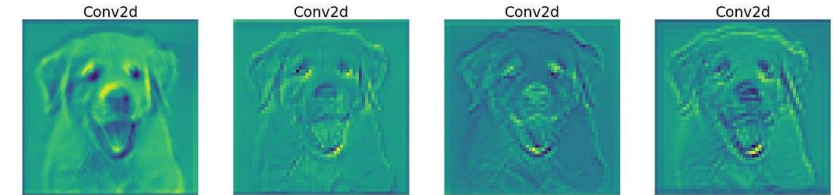
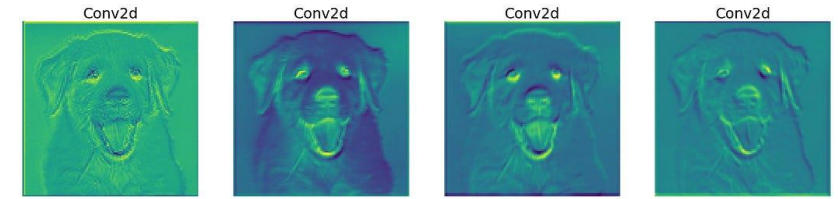
Visualization of Feature Maps

- Learned filters in early layers look like edge detectors.
- Deeper filters: textures, shapes, object parts.
- Visualization = interpretability of CNNs.

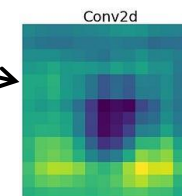


Visualization of Feature Maps

- Learned filters in early layers look like edge detectors.
- Deeper filters: textures, shapes, object parts.
- Visualization = interpretability of CNNs.

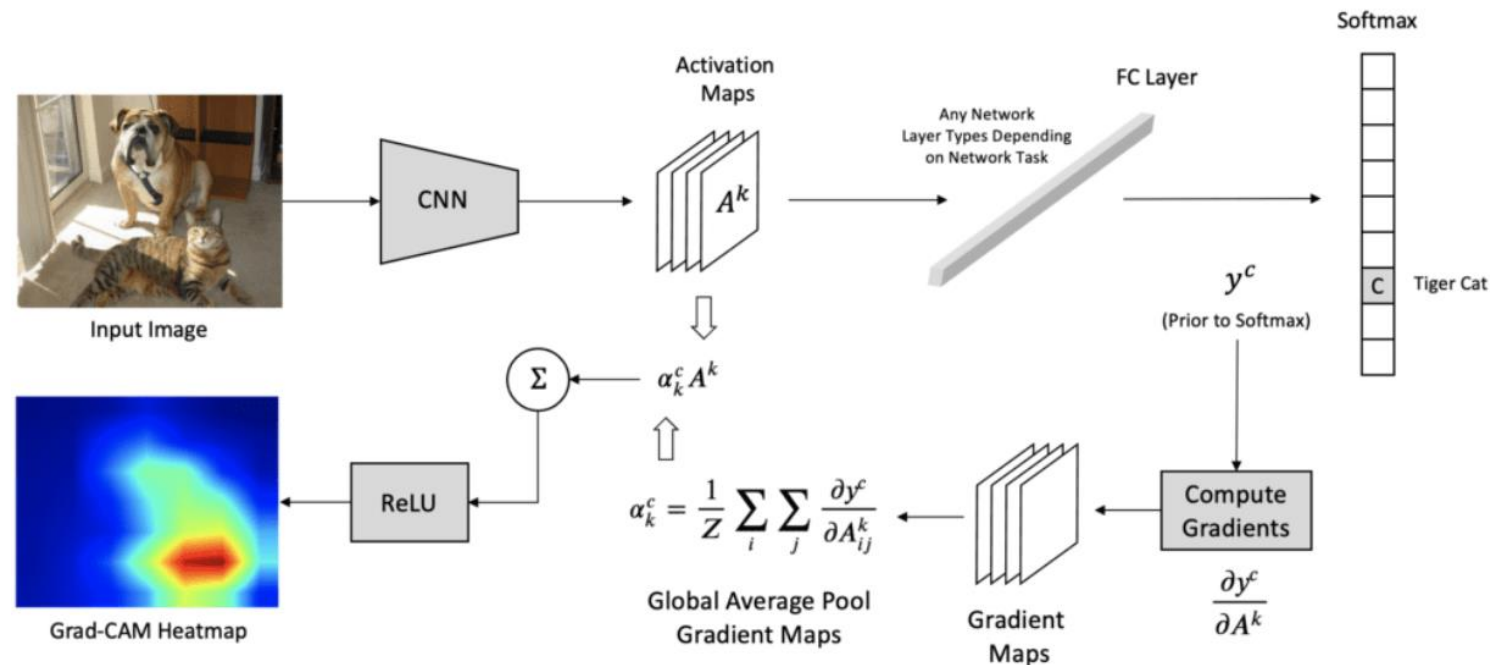


What kind of features do you think this filter detects?



Grad-CAM: Model Explainability

- Gradient-weighted Class Activation Mapping (Grad-CAM) shows regions of input that influenced model's decision.
- Helps understand what CNN 'looks at.'



How Grad-CAM works?

- Take the feature maps from the last convolutional layer.
- Compute the gradients of the target class score with respect to those feature maps.
- Weight each feature map by its gradient and therefore shows how important that map is for that class.
- Combine them into a single heatmap (overlay on the original image).

Modern CNN Architectures

Model	Major Innovation	Year
LeNet-5	Early digit recognition	1998
AlexNet	ReLU + GPU training	2012
VGG	Deep, simple	2014
GoogLeNet	Inception modules	2015
ResNet	Skip connections	2015
EfficientNet	Compound scaling	2019
Vision Transformer (ViT)	Replaces conv with attention	2020

Growth from **5 layers** to **100+ layers**

Regularization in CNNs

- Prevent overfitting via:
 - Dropout
 - Data augmentation
 - Weight decay (L2)
 - Batch normalization
- These make tradeoff between bias-variance and improve generalization.

Limitations of CNNs

- Require large labeled datasets.
- Poor at modeling long-range dependencies.
- Sensitive to adversarial perturbations.
- Struggle with 3D and non-Euclidean data (graphs).
- Transformers are addressing some of these limitations.

Summary of CNN

- A CNN is a combination of two components: feature extraction part and the classification part.
- The convolution + pooling layers perform feature extraction (e.g. two eyes, long ears, four legs, a short tail).
- The fully connected layers then act as a classifier (e.g. assign a probability for the input image being a dog).
- **The convolution layers are the main powerhouse of a CNN model.**

